ML TRAINING - WEEK 2

CALIFORNIA HOUSING PRICE PREDICTOR

Deep learning model tracked with MLflow, served with FastAPI + Docker, and used through a Streamlit app.

Syed Taha Hussaini

Data Science | October 2026



PREDICT | PACKAGE | DEPLOY

THE DATA

Dataset & Preparation

20,640
districts in the California housing data

13
model features (8 numeric + 5 location)

207
missing values in total_bedrooms

$
target: median_house_value

DATA PIPELINE

1 Clean
Filled 207 missing total_bedrooms with the median

2 Encode
One-hot encoded ocean_proximity into 5 columns

3 Split
80/20 train/test split
16,512 train | 4,128 test

4 Scale
StandardScaler fitted on the training data, saved as scaler.pkl

THE MODELS

Three Neural Networks, Six Experiments

Baseline MLP

Simple starting point

- Dense 64 (ReLU)
- Dense 32 (ReLU)
- Output 1

Deeper DNN

More layers, more capacity

- Dense 128 (ReLU)
- Dense 64 (ReLU)
- Dense 32 (ReLU)
- Dense 16 (ReLU)
- Output 1

Regularised DNN

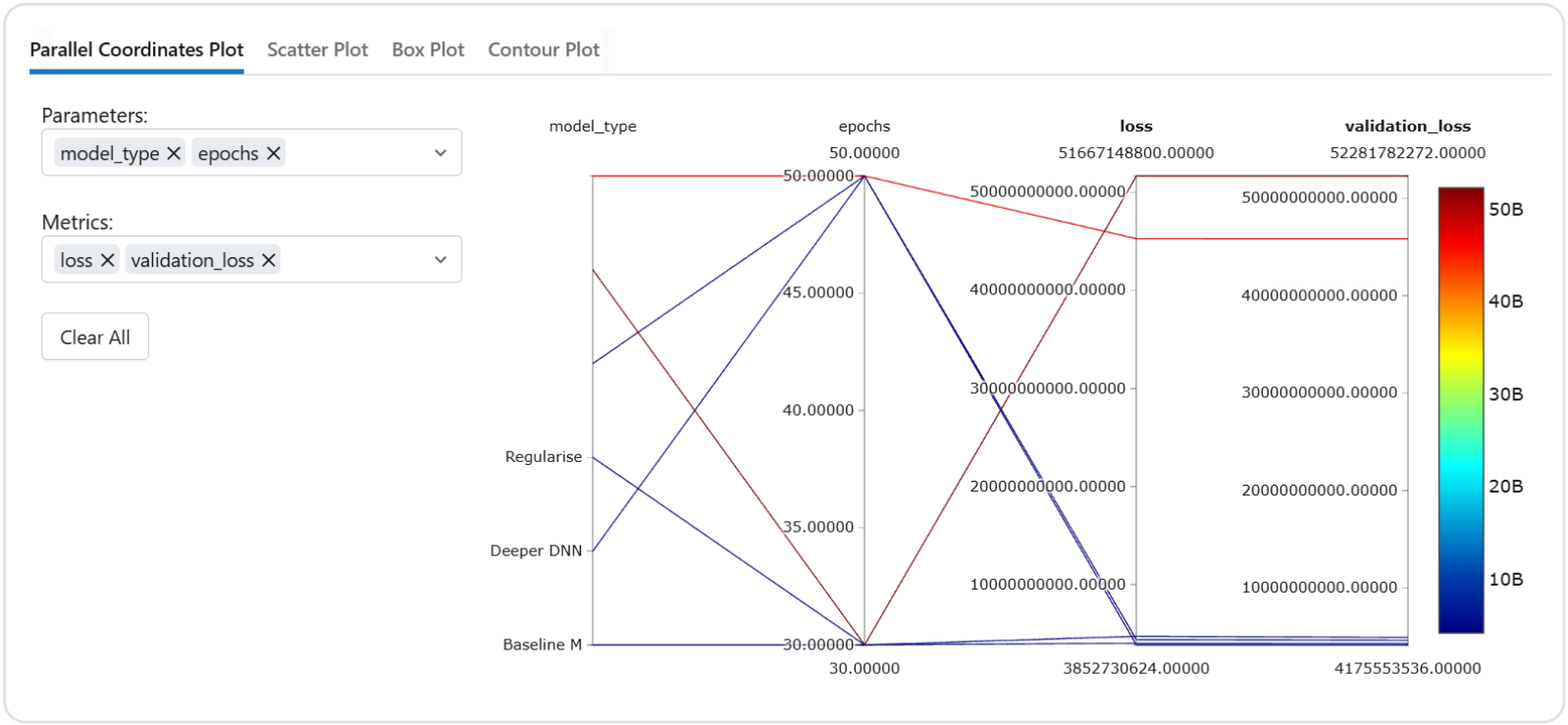
BatchNorm + Dropout

- Dense 64 + BatchNorm
- Dropout 0.1
- Dense 32 + BatchNorm
- Dropout 0.2
- Output 1

OPTIMIZER	LOSS	METRIC	EPOCHS	BATCH SIZE	VALIDATION
Adam	Mean squared error	MAE	30 and 50	30	20% of train

EXPERIMENT TRACKING

Every Run Tracked in MLflow



Parallel coordinates view: model type and epochs against loss and validation loss.

WHAT WE TRACKED

6 runs in one experiment (w2-syed-taha)

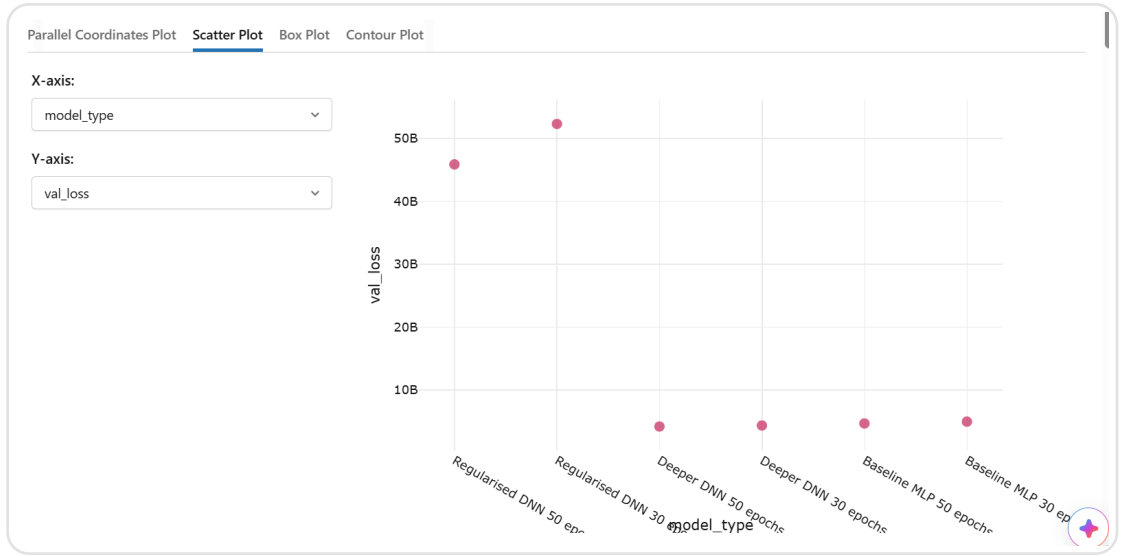
2 parameters: model_type, epochs

3 metrics: loss, val_loss, val_mae

1 best model saved automatically

RESULTS

Deeper DNN Wins



Validation loss by run (lower is better).

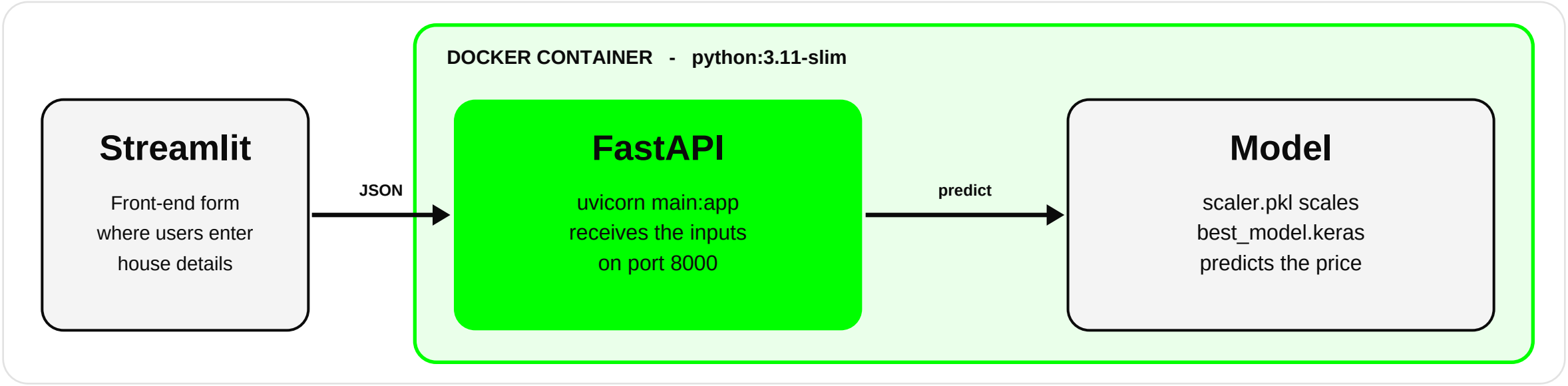
VALIDATION LOSS (MSE, BILLIONS) | VAL MAE ($)

Deeper DNN, 50 epochs	4.18B	**$44.9K**
Deeper DNN, 30 epochs	4.34B	**$46.1K**
Baseline MLP, 50 epochs	4.66B	**$48.3K**
Baseline MLP, 30 epochs	4.95B	**$49.8K**
Regularised, 50 epochs	45.85B	**$188.4K**
Regularised, 30 epochs	52.28B	**$200.3K**

BEST MODEL	VAL LOSS	VAL MAE	VAL RMSE
Deeper DNN, 50 epochs	4.18 billion	~$44.9K	~$64.6K

DEPLOYMENT

From Model to a Running API



RUN IT

```
$ docker build -t taha_image .
$ docker run -d -p 8000:8000 --name taha_container taha_image
```

Same image runs the same way on any machine. Interactive API docs are available at /docs.

THE FRONT-END

Streamlit App



1 **Enter details**

Longitude, latitude, housing age, rooms, bedrooms, population and more.

2 **One click**

The app sends the values to the FastAPI service running in Docker.

3 **Instant price**

The model returns the predicted median house value.

MODEL REGISTRY

Best Model Becomes the Champion

1. **Pick the best run**
   Lowest val_loss, Deeper DNN

2. **Register it**
   Name: housing-price-model

3. **Set the alias**
   champion points to that version

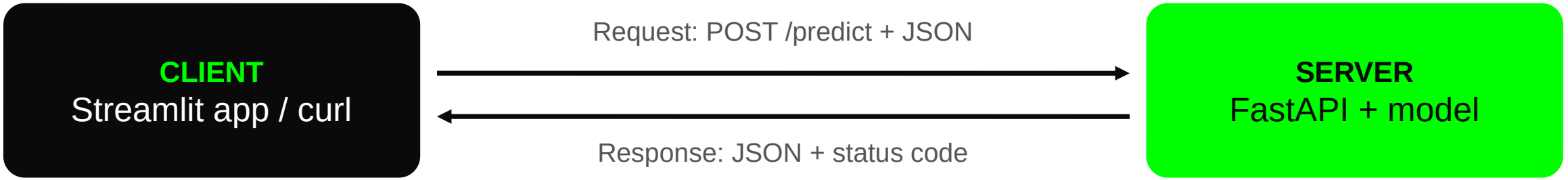
```
mlflow.register_model(
  f"runs:/{best['run_id']}/model",
  "housing-price-model")

client.set_registered_model_alias(
  "housing-price-model",
  "champion", version)
```

The API loads models:/housing-price-model@champion and falls back to best_model.keras if the registry is down.

API BASICS

Client, Server, Request, Response



REST: every resource has its own URL, and the HTTP method says what to do with it.

METHOD	ACTION	IN THIS PROJECT
GET	Read data	**/health, /model-info**
POST	Send data	**/predict**
PUT	Replace	Not needed
DELETE	Remove	Not needed

FASTAPI IN PRODUCTION

- One HTTP layer in front of the model
- Pydantic validates every request
- Auto docs at /docs
- Runs on Uvicorn inside Docker

API ENDPOINTS

Three Endpoints, One Job

GET

/health

Is the service alive? Used for quick checks.

```
{"status": "ok"}
```

GET

/model-info

Which model is loaded, its version and the 13 input features.

```
{"source": "mlflow registry", "alias": "champion", "features": [...]}
```

POST

/predict

Takes 9 house details as JSON and returns the predicted median price.

```
{"predicted_house_price": <float>}
```

Interactive docs for all three: localhost:8000/docs

VALIDATION

Status Codes: What the API Returns

200	**OK**	Valid input. The prediction comes back as JSON.
400	**Bad Request**	Valid JSON but illogical, e.g. bedrooms > rooms.
404	**Not Found**	Unknown URL. FastAPI returns this on its own.
422	**Unprocessable**	Wrong type or range, e.g. latitude "abc" or 500.
500	**Server Error**	Model or scaler fails. Returned as a JSON error.

LIVE TEST

Valid vs Invalid Request

200 OK

```
POST /predict
{
  "longitude": -122.23,
  "latitude": 37.88,
  "housing_median_age": 41,
  ...
  "ocean_proximity": "NEAR BAY"
}

HTTP 200
{"predicted_house_price": <float>}
```

422 Unprocessable

```
POST /predict
{
  "longitude": -122.23,
  "latitude": "abc",
  ...
}

HTTP 422
{"detail": [{"loc": ["body",
  "latitude"], "msg": "Input should
  be a valid number..."}]}
```

Same request, one bad field: the API rejects it before the model ever runs.

WRAP UP

Key Takeaways & Next Steps

WHAT WE BUILT

1. Cleaned and scaled 20,640 housing records
2. Compared 3 architectures x 2 epoch settings
3. Tracked all 6 runs with MLflow
4. Picked the best model: Deeper DNN, 50 epochs
5. Served it with FastAPI inside Docker
6. Built a Streamlit app for end users

NEXT STEPS

Train longer

Regularised models were still improving at 50 epochs.

Tune it

Try learning rates, dropout and layer sizes.

Test set

Report final results on the held-out test data.

Go live

Deploy the Docker image to the cloud.